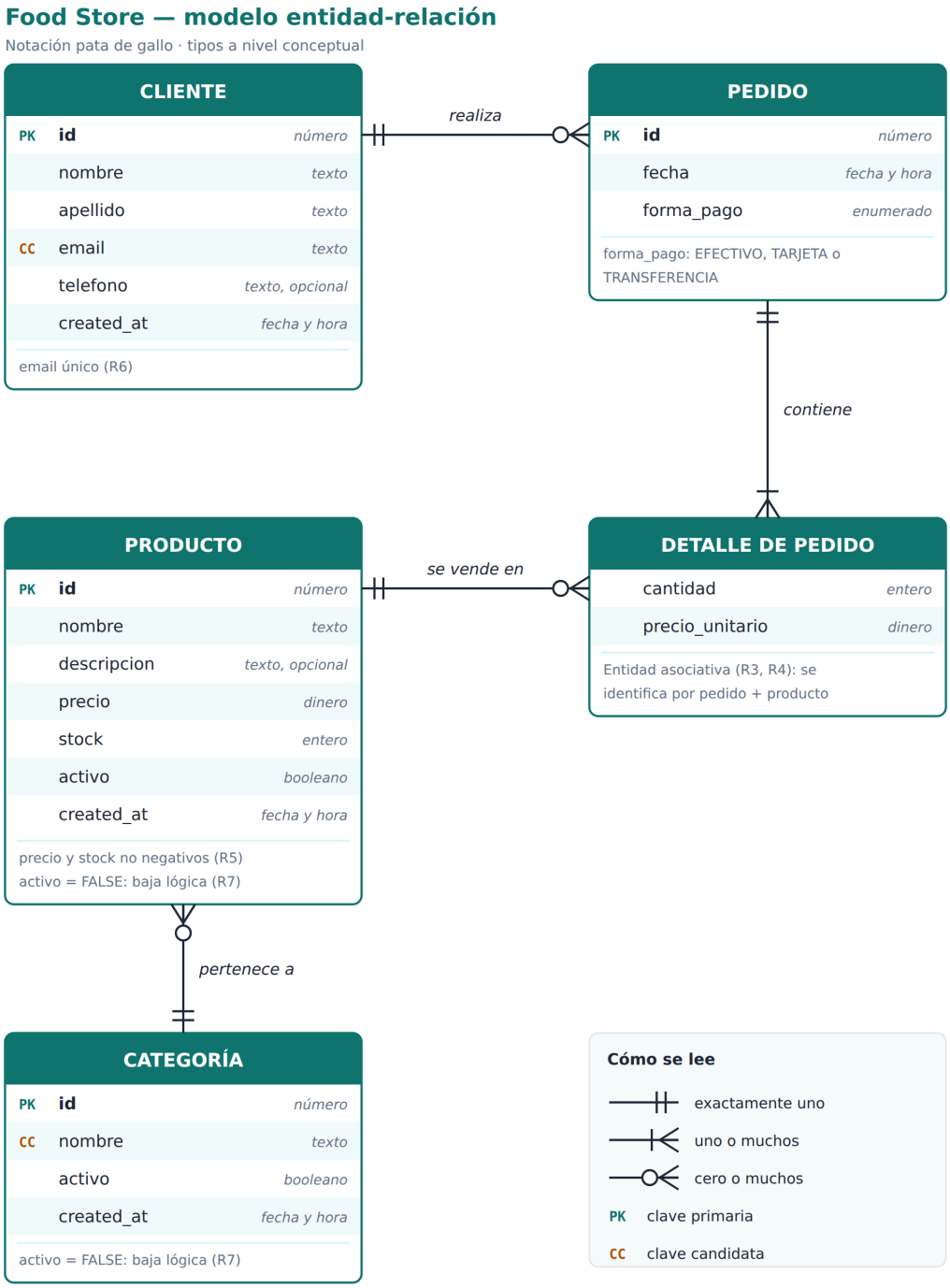
Trabajo Práctico N.º 1 — Food Store

Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Base de Datos II · UTN · Tecnicatura Universitaria en Programación
Moran, Joaquín Leandro

Este documento desarrolla las cuatro partes del trabajo. El DDL de la Parte 4 es el `schema.sql` que se entrega junto con él; al final de este documento está el resumen de sus decisiones, seguido de la declaración de uso de IA. Todo parte del enunciado del negocio y de las reglas R1 a R7 de la consigna.

Parte 1 — Modelo entidad-relación



Notación pata de gallo en todo el diagrama. El símbolo junto a cada entidad indica con cuántas instancias de esa entidad se asocia una instancia de la otra: por ejemplo, «cero o muchos» junto a Pedido dice que un cliente puede tener cero o muchos pedidos.

Diccionario de entidades y atributos

Los tipos son conceptuales: los tipos exactos de PostgreSQL se eligen en la Parte 4. Los atributos marcados como *agregado* no surgen del enunciado; son decisiones propias, como permite la consigna.

Entidad	Atributo	Tipo conceptual	Clave	Observaciones
Categoría	id	número	PK	Identificador que genera el sistema
	nombre	texto	CC	No se repite: dos categorías con el mismo nombre confundirían el menú
	activo	booleano		Baja lógica (R7); verdadero al crearla
	created_at	fecha y hora		*Agregado:* momento del alta
Producto	id	número	PK	Identificador que genera el sistema
	nombre	texto		No se declara único: dos productos de categorías distintas podrían llamarse igual
	descripcion	texto		*Agregado,* opcional
	precio	dinero		Precio de lista vigente; no negativo (R5)
	stock	entero		No negativo (R5); cero al crearlo
	activo	booleano		Baja lógica (R7)
	created_at	fecha y hora		*Agregado*
Cliente	id	número	PK	Identificador que genera el sistema
	nombre	texto		
	apellido	texto		Separado del nombre (tercera pregunta guía)
	email	texto	CC	Identifica al cliente en el sistema (R6)
	telefono	texto		*Agregado,* opcional
	created_at	fecha y hora		*Agregado:* momento del registro
Pedido	id	número	PK	El número de pedido de la planilla
	fecha	fecha y hora		Con zona horaria
	forma_pago	enumerado		EFECTIVO, TARJETA o TRANSFERENCIA: los valores de la planilla
Detalle de pedido	cantidad	entero		Unidades pedidas (R4); mayor que cero
	precio_unitario	dinero		Precio al momento de la venta (R4)

Detalle de pedido no tiene un identificador propio: cada instancia es la combinación de un pedido y un producto (ver la Parte 2).

Relaciones

Relación	Cardinalidad	Participación	Regla
Cliente realiza Pedido	1 cliente : N pedidos	Cliente parcial: puede no haber pedido nunca. Pedido total: siempre tiene cliente.	R2
Pedido contiene Detalle de pedido	1 pedido : N líneas	Pedido total: tiene al menos una línea. Detalle total: toda línea es de un pedido.	R3, R4
Producto se vende en Detalle de pedido	1 producto : N líneas	Producto parcial: puede no haberse vendido. Detalle total: toda línea es de un producto.	R3, R4
Producto pertenece a Categoría	N productos : 1 categoría	Producto total: exactamente una categoría. Categoría parcial: puede estar vacía.	R1

Entre Pedido y Producto queda así una relación N:M (R3), resuelta con la entidad asociativa Detalle de pedido. Esa entidad guarda lo que no es ni del pedido ni del producto, sino de la combinación de los dos: cuántas unidades y a qué precio (R4).

Preguntas guía

¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria 1:N? Porque R3 dice que es «muchos» de los dos lados. Con el pedido del lado «1», cada producto podría estar en un solo pedido: para volver a vender la Muzzarella habría que duplicar el producto, con su precio y su stock. Con el producto del lado «1», cada pedido tendría un solo producto, y el pedido 1 de la planilla (Muzzarella y Coca) no se podría registrar. En los dos casos, además, se pierden la cantidad y el precio unitario de cada línea (R4). No pertenecen al producto, que se vende en cantidades y a precios distintos, ni al pedido, que tiene varios productos: pertenecen al par pedido-producto, y por eso hace falta la entidad asociativa.

¿Qué entidad tiene participación parcial en la relación con categoría y cuál total? ¿Qué pasaría si se invirtiera? Categoría es parcial: puede existir sin productos, como una categoría recién creada (R1). Producto es total: no hay producto sin categoría. Invertida, la lectura obligaría a cargar cada categoría nueva junto con al menos un producto, y a la vez admitiría productos sin categoría. Esos productos quedarían fuera del menú y de todo reporte por categoría, y R1 dejaría de cumplirse.

¿Algún atributo podría descomponerse en partes más simples? El nombre completo del cliente se separó en nombre y apellido: el negocio ordena y busca clientes por apellido y se dirige a ellos por el nombre. Con un solo campo habría que partir el texto cada vez, y con los apellidos compuestos esa partición no sería confiable. Otros atributos se dejaron enteros a propósito:

- El email se usa completo, como identificador (R6).
- El teléfono no se consulta por partes.
- La fecha del pedido es un solo valor de fecha y hora, del que el motor extrae el día o el mes cuando hace falta.

Parte 2 — Modelo relacional

La clave primaria va subrayada; las claves foráneas llevan una flecha hacia la tabla que referencian.

- **categoria** (id, nombre, activo, created_at)
- **producto** (id, nombre, descripcion, precio, stock, activo, created_at, categoria_id → categoria)
- **cliente** (id, nombre, apellido, email, telefono, created_at)
- **pedido** (id, fecha, forma_pago, cliente_id → cliente)
- **detalle_pedido** (pedido_id → pedido, producto_id → producto, cantidad, precio_unitario)

Cómo se derivó.

- Cada entidad fuerte (Categoría, Producto, Cliente y Pedido) es una tabla con su clave primaria `id`.
- Las relaciones 1:N se resuelven con una clave foránea en la tabla del lado N, sin tabla nueva: `categoria_id` en producto (R1) y `cliente_id` en pedido (R2). Las dos son obligatorias (NOT NULL) porque producto y pedido tienen participación total.
- La relación N:M entre pedido y producto (R3) se resuelve con la tabla intermedia `detalle_pedido`. Lleva las dos claves foráneas y los atributos propios de la línea, `cantidad` y `precio_unitario` (R4).
- Las claves candidatas del modelo ER, `categoria.nombre` y `cliente.email`, se conservan como restricciones de unicidad.

Clave de la tabla intermedia: compuesta. La clave primaria de `detalle_pedido` es (`pedido_id`, `producto_id`), sin clave sustituta:

- La regla «un producto no se repite dentro de un pedido» ya convierte ese par en el identificador natural de la línea. Como clave primaria, el motor la hace cumplir sin nada más.
- Ninguna otra tabla referencia una línea de pedido, así que no hace falta una clave corta para apuntarla.
- Con una clave sustituta, igual habría que declarar UNIQUE (`pedido_id`, `producto_id`) para no perder la regla: dos índices en lugar de uno.
- Costo aceptado: si algún día un mismo producto pudiera aparecer dos veces en un pedido (por ejemplo, dos pizzas iguales con distintos agregados), habría que revisar la clave.

Preguntas guía

¿Qué pasaría con la integridad si la tabla intermedia no incluyera las dos claves foráneas como NOT NULL? Podría haber líneas huérfanas:

- Una línea con `pedido_id` nulo sería una venta sin pedido: sin cliente, sin fecha y sin forma de pago.
- Una con `producto_id` nulo sería la venta de algo desconocido, que ningún reporte por producto ni control de stock podría contar.

En este diseño, además, no puede pasar: las dos columnas forman la clave primaria, y PostgreSQL exige NOT NULL en toda columna de una clave primaria.

Si el pedido pudiera existir sin ningún producto, ¿cambia la participación? Sí: la participación de Pedido en la relación con Detalle de pedido pasaría de total a parcial (cero o muchas líneas, en vez de una o muchas). En el DDL no cambia nada, porque la clave foránea está en `detalle_pedido` y apunta a `pedido`: ninguna FK puede exigir que un pedido tenga líneas. «Al menos una línea» se garantiza de otra forma, cargando el pedido y sus líneas en la misma transacción o con un trigger que controle al confirmar. Donde sí impacta es en las consultas: un `JOIN` entre `pedido` y `detalle_pedido` deja afuera los pedidos vacíos. Para listarlos o contarlos hace falta `LEFT JOIN`, y su total da NULL en lugar de cero si no se usa `COALESCE`.

Parte 3 — Normalización hasta 3FN y BCNF

La planilla, tal como la registra el negocio, con una fila por cada línea de producto vendido:

N.º pedido	Fecha	Cliente	Producto	Categoría	Precio unitario	Cant.	Subtotal	Forma de pago
1	01/03/2026	Ana Gómez	Muzzarella	Pizzas	1000.00	2	2000.00	EFECTIVO
1	01/03/2026	Ana Gómez	Coca 1.5L	Bebidas	800.00	1	800.00	EFECTIVO
2	01/03/2026	Luis Paz	Napolitana	Pizzas	1500.00	1	1500.00	TARJETA
3	05/03/2026	Ana Gómez	Muzzarella	Pizzas	1050.00	3	3150.00	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	Coca 1.5L	Bebidas	800.00	4	3200.00	EFECTIVO
4	06/03/2026	Marta Ruiz	Napolitana	Pizzas	1500.00	2	3000.00	EFECTIVO
5	07/03/2026	Luis Paz	Muzzarella	Pizzas	1050.00	1	1050.00	TARJETA

Se toma como relación universal R (nro_pedido, fecha, cliente, producto, categoria, precio_unitario, cantidad, subtotal, forma_pago), con el supuesto de la consigna: en la muestra, el nombre del cliente y el del producto identifican a cada uno.

Paso 1 — Clave candidata

- `nro_pedido` solo no alcanza: el pedido 1 tiene dos filas.
- `producto` solo tampoco: la Muzzarella aparece en los pedidos 1, 3 y 5.
- { `nro_pedido`, `producto` } sí identifica cada fila, porque dentro de un pedido un producto no se repite. Además es mínima: ninguna de sus dos partes alcanza por separado.
- Otras combinaciones no se repiten en la muestra, pero no son claves porque las reglas no lo garantizan. { `nro_pedido`, `categoria` } no lo es, porque un pedido puede llevar dos pizzas distintas. { `fecha`, `cliente`, `producto` } tampoco, porque un cliente puede pedir dos veces el mismo día.

La única clave candidata, y por lo tanto la primaria, es **{nro_pedido, producto}**.

Paso 2 — Dependencias funcionales

Dependencia	Por qué vale
nro_pedido → fecha, cliente, forma_pago	Cada pedido tiene una única fecha, un único cliente y una única forma de pago
producto → categoria	Cada producto pertenece a una única categoría
{nro_pedido, producto} → cantidad, precio_unitario	Son los datos de una línea del pedido
{cantidad, precio_unitario} → subtotal	subtotal = cantidad × precio_unitario

Y las que **no** valen, aunque la muestra pueda sugerirlas:

- **producto → precio_unitario no vale:** la Muzzarella se vendió a 1000.00 en el pedido 1 y a 1050.00 en los pedidos 3 y 5. El precio de la línea depende del producto *y* del pedido, porque es el precio del momento de la venta (R4). Esta es la observación clave de la pista.
- **cliente → forma_pago no vale:** Ana Gómez pagó en efectivo el pedido 1 y por transferencia el 3.
- **fecha → cliente no vale:** el 01/03/2026 compraron Ana Gómez y Luis Paz.

Paso 3 — Primera forma normal

Cumple. Cada celda tiene un solo valor (no hay listas del tipo «Muzzarella x2, Coca x1»), no hay grupos repetidos y cada fila se identifica por la clave del paso 1. Por el supuesto de la consigna, el nombre del cliente se toma como un valor atómico.

Paso 4 — Segunda forma normal

No cumple, porque hay dependencias parciales de la clave:

- `fecha`, `cliente` y `forma_pago` dependen solo de `nro_pedido`.
- `categoria` depende solo de `producto`.

En la planilla se ve el efecto. La fecha, el cliente y la forma de pago del pedido 1 se repiten en sus dos filas, y que la Muzzarella es una pizza está escrito tres veces. Eso trae tres anomalías:

- **Actualización:** cambiar la forma de pago de un pedido obliga a tocar todas sus filas.
- **Inserción:** no se puede registrar un producto que todavía no se vendió.
- **Borrado:** si se borran los pedidos de un producto, se pierde a qué categoría pertenece.

Cada dependencia parcial se separa en su propia tabla:

- **pedido** (nro_pedido, fecha, cliente, forma_pago)
- **producto** (producto, categoria)
- **detalle** (nro_pedido, producto, cantidad, precio_unitario, subtotal)

`precio_unitario` se queda en detalle porque depende de la clave completa (paso 2).

Paso 5 — Tercera forma normal

- **pedido:** `fecha`, `cliente` y `forma_pago` dependen directamente de `nro_pedido`, y ninguno determina a otro (paso 2). Cumple.
- **producto:** tiene un solo atributo no clave. Cumple.
- **detalle:** no cumple. `subtotal` depende de `cantidad` y `precio_unitario`, que no son clave: {nro_pedido, producto} → {cantidad, precio_unitario} → subtotal es una dependencia transitiva.

Se resuelve eliminando `subtotal`. No hace falta una tabla nueva, porque es un cálculo sobre la misma fila (ver la última pregunta de integración).

Paso 6 — BCNF

Todo determinante tiene que ser clave candidata:

Tabla	Determinantes	¿Es clave candidata?
pedido	nro_pedido	Sí
producto	producto	Sí
detalle	{nro_pedido, producto}	Sí

No hay excepciones. En detalle podría pensarse en {nro_pedido, precio_unitario} → producto, porque en la muestra no hay dos productos con el mismo precio dentro de un pedido. Pero no es una regla del negocio: dos pizzas de 1500.00 podrían ir en el mismo pedido.

BCNF solo se separa de 3FN cuando un determinante que no es clave determina parte de una clave, y eso requiere claves candidatas que se superpongan. Acá cada tabla tiene una sola clave candidata, así que estar en 3FN ya implica estar en BCNF.

Paso 7 — Tablas finales

Con la clave primaria subrayada y los datos de la planilla:

pedido

nro_pedido	fecha	cliente	forma_pago
1	01/03/2026	Ana Gómez	EFECTIVO
2	01/03/2026	Luis Paz	TARJETA
3	05/03/2026	Ana Gómez	TRANSFERENCIA
4	06/03/2026	Marta Ruiz	EFECTIVO
5	07/03/2026	Luis Paz	TARJETA

producto

producto	categoria
Muzzarella	Pizzas
Coca 1.5L	Bebidas
Napolitana	Pizzas

detalle

nro_pedido	producto	cantidad	precio_unitario
1	Muzzarella	2	1000.00
1	Coca 1.5L	1	800.00
2	Napolitana	1	1500.00
3	Muzzarella	3	1050.00
4	Coca 1.5L	4	800.00
4	Napolitana	2	1500.00
5	Muzzarella	1	1050.00

Preguntas de integración

¿A qué entidad del modelo ER corresponde cada tabla? `pedido` corresponde a la entidad Pedido; `producto`, a Producto; `detalle`, a Detalle de pedido. En la planilla, el cliente y la categoría son simples nombres, pero en el ER son entidades con clave propia, porque el negocio necesita guardar más datos de ellos. En la base real, la columna `cliente` de pedido pasa a ser `cliente_id`, y la columna `categoria` de producto pasa a ser `categoria_id`.

¿Qué atributos del ER no aparecían en la planilla y qué dice eso? No aparecían el email y el teléfono del cliente, ni el apellido por separado; tampoco la descripción, el precio de lista y el stock del producto, ni las marcas `activo` y las fechas de alta. La planilla se armó para facturar y guarda solo lo que hacía falta para eso. Normalizar datos históricos revela la estructura que ya tienen (acá, que el precio pertenece a la línea y no al producto), pero no puede agregar lo que nunca se registró. Modelar desde el enunciado del negocio agrega lo que el sistema necesita para operar (R5, R6 y R7). Los dos caminos se complementan, y el esquema de la Parte 4 concilia ambos.

¿Conviene almacenar el subtotal? No. Guardarlo vuelve a introducir la dependencia transitiva que se quitó en el paso 5, con el riesgo de que quede desactualizado. Igual, la decisión de fondo no es de formas normales sino de rendimiento o trazabilidad, y acá ninguna lo justifica: el subtotal es una multiplicación

sobre la misma fila, y la trazabilidad ya la da `precio_unitario`, congelado al vender (R4). Si un reporte lo necesitara precalculado, una columna generada (`GENERATED ALWAYS AS (cantidad * precio_unitario) STORED`) lo mantendría siempre consistente.

Parte 4 — Decisiones del DDL

El script completo es `schema.sql`, con un comentario en cada decisión que no es obvia. En resumen:

Decisión	Dónde	Por qué
`BIGINT GENERATED ALWAYS AS IDENTITY`	Clave `id` de las tablas fuertes	Clave autogenerada; ALWAYS impide cargar un id a mano
`NUMERIC(10,2)`	precio, precio_unitario	Dinero con precisión fija, sin errores de redondeo
`ENUM tipo_forma_pago`	pedido.forma_pago	Dominio cerrado: el motor rechaza cualquier otro valor
`TIMESTAMPTZ` con `DEFAULT now()`	pedido.fecha y los created_at	Fecha y hora con zona horaria, cargadas al insertar
`ON DELETE RESTRICT`	producto → categoria, pedido → cliente, detalle → producto	Lo que tiene ventas o productos no se borra: se da de baja (R7)
`ON DELETE CASCADE`	detalle → pedido	Una línea no tiene sentido sin su pedido
`CHECK`	precio, stock, cantidad, precio_unitario y email	R5 y R4; el email en minúsculas y con arroba
`UNIQUE`	categoria.nombre, cliente.email	Las claves candidatas del modelo ER (R6)
Índices	cliente de un pedido, fecha del pedido, productos activos por categoría, producto del detalle	Cada uno con la consulta que acelera, comentada en el script

El email se guarda en minúsculas (CHECK) para que la restricción UNIQUE no acepte el mismo correo escrito con otras mayúsculas.

Declaración de Uso de IA (DUIA)

Campo	Detalle
Herramienta	Claude (Anthropic), en conversación
Para qué	Desarrollar las Partes 1 a 3, dibujar el diagrama entidad-relación y escribir el `schema.sql` de la Parte 4
Insumos	La consigna de este trabajo: el enunciado del negocio, las reglas R1 a R7 y la planilla de la Parte 3
Prompts	«bueno pero tienen que ser todos los trabajos de todas las unidades» y «no pero me dijiste que hibamos a hacer todo nosotros no del otro repo grupal»

Qué generó la IA

- El diccionario de entidades y atributos, las relaciones con su cardinalidad y participación, y las respuestas a las preguntas guía de las Partes 1 y 2.

- El diagrama en notación pata de gallo, con los tipos a nivel conceptual.
- El modelo relacional y la justificación de la clave compuesta de `detalle_pedido`.
- Los siete pasos de la normalización, con sus dependencias funcionales, y las respuestas a las preguntas de integración.
- El `schema.sql`, con sus comentarios.

Qué se aceptó y qué se modificó

- **Aceptado:** todo lo generado, después de revisarlo.
- **Modificado:** nada por ahora. Si en la revisión o en la defensa cambio alguna decisión, queda registrada acá.

Verificación

- El `schema.sql` se ejecutó sin errores en mi PC con PostgreSQL 18.6: crea la base de los trabajos de la Unidad 1 (`scripts/crear_base.sh` de la Semana 2, evidencia `00_base.txt`).
- En el entorno de Claude, con PostgreSQL 16, el script se ejecutó dos veces seguidas sin errores y se probaron casos inválidos. El motor rechazó todos: categoría repetida, precio negativo, producto sin categoría, email con mayúsculas o sin arroba, forma de pago fuera del ENUM, producto repetido en un pedido, cantidad cero, y el borrado de una categoría, un cliente o un producto que tienen filas que los referencian.
- El diagrama, el modelo relacional y el `schema.sql` tienen las mismas tablas, atributos, claves y cardinalidades.